DATA STRUCTURES & ALGORITHMS · C++

Heaps & Priority Queues in C++ From One Array to Placement-Ready

Everything for heaps and priority queues: how a heap lives inside an array, sift-up and sift-down by hand, O(n) heapify and heap sort, std::priority_queue and its comparator trap, top-K, k-way merge, two heaps, greedy scheduling, Dijkstra, the 12 patterns, and a curated problem list.

45	12	15	38
SECTIONS	PATTERNS	CODE BLOCKS	PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Generated 14 October 2026

Contents

PART 0 — Priority queues and heaps, from absolute zero

0.1 Why not just keep a sorted array?

0.2 What a heap is

0.3 The array representation — why heaps need no pointers

0.4 The vocabulary

0.5 How to recognise the topic in a problem statement

PART 1 — Building a heap by hand

1.1 Sift up — how push works

1.2 Sift down — how pop works

1.3 Build-heap in O(n) — and why it is not O(n log n)

1.4 Heap sort

1.5 A min-heap, and other variations

PART 2 — The STL: std::priority_queue

2.1 The basics

2.2 The comparator trap — read this twice

2.3 Custom comparators — three ways

2.4 The algorithm functions

2.5 What a heap cannot do, and the two workarounds

PART 3 — The heap mental model

3.1 "The heap holds the candidates; the top is the next decision"

3.2 The size-k rule — the inversion everyone gets wrong once

3.3 Heap, sort, BST or quickselect?

3.4 Push indices, or push enough to find your way back

PART 4 — The Top-K family

4.1 K-th largest element (LC 215) — three ways

4.2 Top K frequent (LC 347, LC 692)

4.3 K closest points (LC 973) and K closest elements (LC 658)

4.4 K-th largest in a stream (LC 703)

PART 5 — K-way merge

5.1 The idea

5.2 Problems that are secretly k-way merges

5.3 Smallest range (LC 632) — the k-way merge with a window

PART 6 — Two heaps

6.1 Median of a data stream (LC 295)

6.2 Sliding window median (LC 480)

6.3 Two pools: locked and available

PART 7 — Greedy and scheduling with a heap

7.1 Repeatedly combine the two best

7.2 Take the largest, put back what is left

7.3 Arrange so that no two neighbours are equal

7.4 The "regret" pattern — take now, undo the worst later

7.5 Simulating machines and time

PART 8 — Heaps on graphs and grids

8.1 Dijkstra's algorithm

8.2 Minimax paths — "the cost of a path is its worst step"

8.3 Trapping Rain Water II (LC 407)

PART 9 — The complete pattern taxonomy

How to tell 2 and 4 apart

How to tell 6 and 8 apart

PART 10 — How to approach ANY heap problem

Step 1 — Ask whether you need "the best", repeatedly, while the set changes

Step 2 — Decide what one heap entry is

Step 3 — Decide the direction, out loud

Step 4 — Bound the heap

Step 5 — Handle the clock and the empty heap

Step 6 — Dry-run the small cases

PART 11 — The bug catalogue

PART 12 — Complexity cheat sheet

PART 13 — The problem list

PART 14 — A three-week plan

Quick reference card

PART 0 — Priority queues and heaps, from absolute zero

A stack hands back the **newest** element. A queue hands back the **oldest**. A **priority queue** hands back the **most important** one — the largest, the smallest, the earliest deadline, the closest point — no matter when it arrived.

That is the whole abstract idea. Three operations:

OPERATION	MEANING
`push(x)`	add an element
`top()`	look at the most important element
`pop()`	remove the most important element

A **heap** is the data structure almost everyone uses to implement a priority queue, because it does `push` and `pop` in O(log n) and `top` in O(1) — and it does it inside a plain array, with no pointers.

0.1 Why not just keep a sorted array?

IMPLEMENTATION	PUSH	TOP	POP
unsorted array	O(1)	O(n)	O(n)
sorted array	O(n) — shift to insert	O(1)	O(1)
balanced BST (`std::set`)	O(log n)	O(log n) / O(1)	O(log n)
binary heap	**O(log n)**	**O(1)**	**O(log n)**

A sorted array does too much work: it keeps **every** element in order, but a priority queue only ever needs to know **the best one**. A heap keeps just enough order to answer that one question, and that is why it is cheaper.

> **The single sentence to remember:** a heap is *partially* sorted — every parent beats its children, and nothing else is promised. That weak promise is enough to find the best element instantly and to repair the structure in O(log n).

0.2 What a heap is

A **binary heap** is a binary tree with two properties:

1. **Shape property** — it is a **complete** binary tree: every level is full except possibly the last, and the last level is filled **from the left**.
2. **Heap property** — in a **max-heap** every parent is **≥** its children; in a **min-heap** every parent is **≤** its children.

So the root is always the maximum (max-heap) or minimum (min-heap).

```
a max-heap           NOT a heap: 15 and 10 beat their parent 9

   90                     9
  ┌──┐                  ┌──┐
 15   10              15    10
┌──┐   │             ┌──┐
7  12  2             7  12
```

The heap property says nothing about left versus right, or about cousins. `15` and `10` are in no particular order. `12` is bigger than `10` even though it sits lower. That is fine.

0.3 The array representation — why heaps need no pointers

Because the tree is complete, you can number the nodes level by level, left to right, and store them in an array with **no gaps**:

```
          90 [0]
        ┌──────┐
   15 [1]       10 [2]
  ┌──┐  │        │
7 [3]   12 [4]   2 [5]

index:   0    1    2    3    4    5
array: [ 90 | 15 | 10 |  7 | 12 |  2 ]
```

The tree edges become arithmetic. For a **0-indexed** array:

```
parent(i) = (i - 1) / 2
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2
```

Check it: node 4 (value 12) has parent (4 − 1) / 2 = 1 (value 15). Node 1 has children 3 and 4.

Some textbooks and most GfG articles use **1-indexed** arrays (leave index 0 empty), where the formulas become `parent = i / 2`, `left = 2i`, `right = 2i + 1`. Both are fine; **never mix them**. This guide uses 0-indexing throughout.

Two consequences of the shape property that you will use constantly:

- A heap of n elements has height **⌊log₂ n⌋**, so anything that walks one root-to-leaf path is O(log n).
- The **leaves** are exactly indices ⌊n/2⌋ … n − 1 — the second half of the array. The last internal node is **n/2 − 1**.

0.4 The vocabulary

TERM	MEANING
Priority queue	The abstract type: push, top, pop-the-best
Heap	The usual implementation: a complete binary tree with the heap property
Max-heap / min-heap	Root is the largest / the smallest
Sift up (bubble up, percolate up, heapify-up)	Move an element up while it beats its parent — used by `push`
Sift down (bubble down, heapify-down, `heapify(i)`)	Move an element down while a child beats it — used by `pop`
Heapify / build-heap	Turn an arbitrary array into a heap, in O(n)
Heap sort	Build a max-heap, then repeatedly move the root to the end
Comparator	The function that decides which element is "more important"
Lazy deletion	Leave an unwanted element inside the heap and skip it when it reaches the top
K-way merge	Merge k sorted sequences by keeping one candidate from each in a heap

0.5 How to recognise the topic in a problem statement

You will rarely be told "use a heap." You get these signals instead:

THE PROBLEM SAYS	REACH FOR
"k-th largest / smallest", "top k", "k most frequent", "k closest"	a heap of size k
"merge k sorted …", "k-th smallest in a sorted matrix", "smallest pairs"	k-way merge with a min-heap
"median of a stream", "running median"	two heaps
"repeatedly take the largest / smallest and …"	a heap, simulated
"minimum cost to combine", "connect ropes", Huffman	min-heap greedy
schedule tasks / meetings / CPU / servers, "earliest available"	heap of free times
shortest path with non-negative weights	Dijkstra = BFS with a min-heap
"at each step choose the best available option" with options unlocking over time	sort + heap
a stream where you only care about the best few elements seen so far	bounded heap

The constant theme: **you need the best element, many times, while the set keeps changing.** If you only need it once, sort or scan. If the set never changes, sort once. A heap earns its place when inserts and extractions are interleaved.

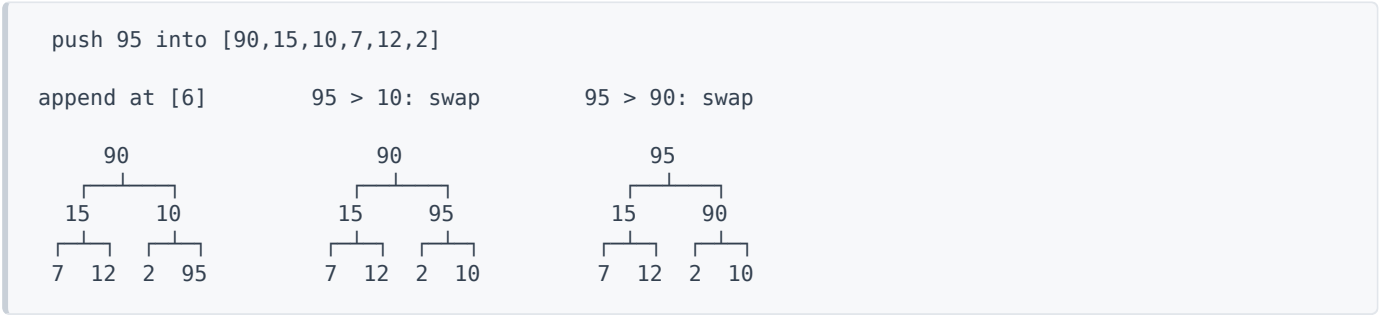
PART 1 — Building a heap by hand

Interviewers ask you to implement a heap surprisingly often, and Indian written rounds ask for heapify, heap sort and "is this array a heap?" by hand. Write each of these once and you will never be caught out.

1.1 Sift up — how push works

To push, put the new element in the **only** place that keeps the tree complete: the end of the array. It may now beat its parent, so swap it upward until it does not.

```
void siftUp(vector<int>& a, int i) {
    while (i > 0 && a[(i - 1) / 2] < a[i]) {     // bigger than my parent?
        swap(a[(i - 1) / 2], a[i]);
        i = (i - 1) / 2;
    }
}
void push(vector<int>& a, int x) {
    a.push_back(x);
    siftUp(a, (int)a.size() - 1);
}
```



At most one swap per level: **O(log n)**.

1.2 Sift down — how pop works

The root is the answer, so remove it. The hole must be filled while keeping the tree complete, so move the **last** element into the root. It is probably too small for the top, so swap it down with its **larger** child until it beats both children.

```
void siftDown(vector<int>& a, int n, int i) {     // heap is a[0..n-1]
    while (true) {
        int largest = i, l = 2 * i + 1, r = 2 * i + 2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest == i) return;                  // both children are smaller
        swap(a[i], a[largest]);
        i = largest;
    }
}
void pop(vector<int>& a) {
    a[0] = a.back();
    a.pop_back();
    if (!a.empty()) siftDown(a, (int)a.size(), 0);
}
```

```
 pop from [95,15,90,7,12,2,10]

remove 95            last (10) to the root        10 < 90: swap. done

     95                     10                          90
   ┌──┴──┐                ┌──┴──┐                     ┌──┴──┐
  15     90              15    90                    15    10
 ┌─┴─┐  ┌─┴─┐           ┌─┴─┐   │                   ┌─┴─┐   │
 7  12  2  10          7  12  2                     7  12  2
```

Why the larger child? After the swap, the child you promoted becomes the parent of its old sibling. It is only ≥ that sibling if you picked the larger of the two. Swapping with the smaller child breaks the heap property one level up — this is bug #2 in the catalogue.

The parameter `n` lets sift-down work on a **prefix** of the array, which heap sort needs.

1.3 Build-heap in O(n) — and why it is not O(n log n)

You can build a heap by pushing n elements one by one: O(n log n). There is a better way:

```cpp
void buildMaxHeap(vector<int>& a) {
    int n = a.size();
    for (int i = n / 2 - 1; i >= 0; i--)   // every internal node, bottom-up
        siftDown(a, n, i);
}
```

A full trace on the GfG example array (n = 11, so the last internal node is index 4):

```
 i   sift down a[i]   array after
 -   start            [1, 3, 5, 4, 6, 13, 10, 9, 8, 15, 17]
 4   a[4] = 6         [1, 3, 5, 4, 17, 13, 10, 9, 8, 15, 6]
 3   a[3] = 4         [1, 3, 5, 9, 17, 13, 10, 4, 8, 15, 6]
 2   a[2] = 5         [1, 3, 13, 9, 17, 5, 10, 4, 8, 15, 6]
 1   a[1] = 3         [1, 17, 13, 9, 15, 5, 10, 4, 8, 3, 6]
 0   a[0] = 1         [17, 15, 13, 9, 6, 5, 10, 4, 8, 3, 1]

 the result as a tree:

         17
      ┌──┴──┐
     15     13
    ┌─┴─┐  ┌─┴─┐
    9   6  5  10
   ┌┴┐ ┌┴┐
   4 8 3 1
```

Leaves are already one-element heaps, so start at the last internal node and sift each node down. When you reach node i, both its subtrees are heaps already, so one sift-down fixes the subtree at i.

Why O(n): sift-down from node i costs its **height**, not log n. Half the nodes are leaves (height 0, no work), a quarter have height 1, an eighth height 2, …

```
total work ≈  n/4 · 1  +  n/8 · 2  +  n/16 · 3  +  …   =  n · Σ h / 2^(h+1)   ≤  n
```

The many nodes near the bottom do almost nothing; only the few near the top travel far. This is a standard interview question — "*what is the complexity of heapify, and why?*" — and the answer is **O(n)** with exactly that argument.

`std::priority_queue<int> pq(v.begin(), v.end());` uses this O(n) build. Pushing in a loop does not.

1.4 Heap sort

Build a max-heap; the maximum is at index 0. Swap it with the last element — it is now in its final sorted position — shrink the heap by one, and sift the new root down. Repeat.

```
void heapSort(vector<int>& a) {
    buildMaxHeap(a);
    for (int end = (int)a.size() - 1; end > 0; end--) {
        swap(a[0], a[end]);        // max goes to its final slot
        siftDown(a, end, 0);       // the heap is now a[0..end-1]
    }
}
```

	HEAP SORT	MERGE SORT	QUICK SORT
Worst case	**O(n log n)**	O(n log n)	O(n²)
Extra space	**O(1)**	O(n)	O(log n)
Stable	no	yes	no
Cache behaviour	poor (jumps around)	good	very good

Heap sort is the only one of the three with O(n log n) worst case **and** O(1) space. In practice it loses to quicksort because of cache misses — `std::sort` (introsort) falls back to heap sort only when quicksort recursion gets too deep.

1.5 A min-heap, and other variations

A min-heap is the same code with every comparison flipped. Four other things worth knowing at interview depth:

- **Is this array a heap?** One loop: for every i ≥ 1, check `a[parent(i)] >= a[i]`.
- **Decrease-key / increase-key** — change a value in place, then sift up or down from its index. It needs to know *where* an element is, so real implementations keep a position map. `std::priority_queue` does not support it; Part 2.5 shows the standard workaround.
- **Delete an arbitrary element at index i** — move the last element into slot i, shrink, then sift **both** up and down (only one will do anything).
- **d-ary heaps** have d children per node: shorter trees, cheaper pushes, more expensive pops. Rarely asked; know that they exist.

PART 2 — The STL: `std::priority_queue`

2.1 The basics

```
#include <queue>

priority_queue<int> maxH;                                  // max-heap (default)
priority_queue<int, vector<int>, greater<int>> minH;       // min-heap

maxH.push(5);   maxH.emplace(7);
maxH.top();     // 7 — the LARGEST
maxH.pop();     // returns void, like stack::pop
maxH.size();    maxH.empty();

priority_queue<int> fromVec(v.begin(), v.end());           // O(n) heapify
```

OPERATION	COST
`push`, `emplace`, `pop`	O(log n)
`top`, `size`, `empty`	O(1)
construct from a range	O(n)
find / erase an arbitrary element	**not supported**
iterate in order	**not supported**

The same two C++ facts from Stacks & Queues apply: **`pop()` returns `void`**, and **`top()` on an empty heap is undefined behaviour.** Guard every access.

2.2 The comparator trap — read this twice

The third template parameter is a comparator `cmp(a, b)`. The rule is:

> **`cmp(a, b)` returns `true` when `a` should sit BELOW `b`** — that is, when `a` has *lower* priority. The element for which `cmp` is never true against anything ends up on top.

So the default `less<int>` (`a < b`) puts the **largest** on top — a max-heap — and `greater<int>` puts the **smallest** on top. It is backwards compared to `std::sort`, where `less` gives ascending order with the smallest **first**.

```
std::sort(v, less)            -> smallest first
priority_queue<.., less>      -> LARGEST on top     (the opposite!)
priority_queue<.., greater>   -> smallest on top
```

The way to stop getting this wrong: **the heap's top is the element that would come LAST if you sorted with the same comparator.**

2.3 Custom comparators — three ways

```cpp
// 1. A functor (struct with operator()) — works everywhere, clearest
struct ByDist {
    bool operator()(const Point& a, const Point& b) const {
        return a.dist > b.dist;        // '>' -> min-heap on dist
    }
};
priority_queue<Point, vector<Point>, ByDist> pq1;

// 2. A lambda — needs decltype and the lambda passed to the constructor
auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq2(cmp);

// 3. Encode priority in a pair/tuple and use the built-in ordering
priority_queue<pair<int, int>> pq3;                          // max by first, then second
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq4;  // min-heap
```

Pairs and tuples compare lexicographically: first by `.first`, ties by `.second`, and so on. That makes option 3 the fastest to write in an interview. Put the priority first and the payload (an index, a node id) second: `{distance, node}`, `{frequency, value}`, `{value, row, col}`.

Watch the tie-break you get for free: in a max-heap of `pair<int,int>`, equal firsts are ordered by the **larger** second on top. When a problem specifies a tie-break (LC 692 — alphabetical order; LC 1337 — smaller index), check that the free one is the one you want, or write a functor.

A negation shortcut for a min-heap of numbers: push `-x` into a max-heap and negate on the way out. It works, it is common in contest code, and it breaks on `INT_MIN`. Prefer `greater<>`.

2.4 The algorithm functions

`<algorithm>` exposes the heap operations directly on a vector:

```cpp
make_heap(v.begin(), v.end());          // O(n) heapify, max-heap
v.push_back(x); push_heap(v.begin(), v.end());   // sift the last element up
pop_heap(v.begin(), v.end()); v.pop_back();      // moves max to the back, then remove
sort_heap(v.begin(), v.end());          // heap sort the range
is_heap(v.begin(), v.end());            // the "is this a heap?" check
```

Useful when you need to *see* the heap's contents (a `priority_queue` hides its container) or when a problem gives you the array and asks for heap operations on it.

2.5 What a heap cannot do, and the two workarounds

`std::priority_queue` cannot **erase an arbitrary element** and cannot **change a priority** in place. Many problems need one of those. Two standard answers:

Workaround 1 — lazy deletion. Do not remove the element; remember that it is dead, and throw it away when it reaches the top.

```cpp
// Dijkstra: instead of decrease-key, push a new (dist, node) pair.
// Old pairs for the same node become stale; skip them when popped.
auto [d, u] = pq.top(); pq.pop();
if (d > dist[u]) continue;              // stale entry — already improved
```

The heap may grow larger than the live set, but each stale entry is popped once, so the total cost is unchanged up to a constant. The Skyline problem (LC 218) uses the same trick: buildings whose right edge has passed are discarded only when they surface.

Workaround 2 — use `std::multiset` as a heap. A balanced BST gives min (`*s.begin()`), max (`*s.rbegin()`), insert and **erase of any element** in O(log n). It is slower by a constant factor and uses more memory, but it supports everything. Sliding Window Median (LC 480) is the textbook case: the window must forget specific old elements.

NEED	USE
only push / top / pop	`priority_queue`
also erase a known value	`multiset` (erase **one** copy with `s.erase(s.find(x))`)
change priorities, many stale entries are fine	`priority_queue` + lazy deletion
min **and** max at once	`multiset`, or two heaps

> `s.erase(x)` on a multiset erases **every** copy of x. To remove one, erase the iterator: `s.erase(s.find(x))`. This is a guaranteed wrong answer the first time you meet it.

PART 3 — The heap mental model

3.1 "The heap holds the candidates; the top is the next decision"

Every heap solution has the same shape. There is a pool of **candidates** — elements still in play — and at each step you need **the best candidate right now**. The heap holds the pool. The top is the decision.

Before writing code, answer two questions:

1. **What is a candidate, and what goes into the heap for it?** A value? `(priority, index)`? `(value, row, col)`? Decide the tuple first.
2. **What makes one candidate better than another?** That is the comparator — and it decides min-heap versus max-heap.

Then the code is a loop: *pop the best, act on it, push whatever it unlocks.*

3.2 The size-k rule — the inversion everyone gets wrong once

To keep the **k largest** elements of a stream, use a **min-heap** of size k. To keep the **k smallest**, use a **max-heap** of size k.

That feels backwards. Here is why it is right: the heap of size k is a **club of the best k seen so far**, and its top must be the member **most likely to be thrown out**. When a new element arrives, it is compared against the *weakest* member — the smallest of the k largest — so the weakest must be on top.

```
keep the 3 largest of  5, 1, 9, 3, 7, 8    (min-heap, capacity 3)

push 5   {5}
push 1   {1,5}
push 9   {1,5,9}
push 3   {1,3,5,9}  size 4 > 3: pop the min (1)  ->  {3,5,9}
push 7   {3,5,7,9}  pop 3                        ->  {5,7,9}
push 8   {5,7,8,9}  pop 5                        ->  {7,8,9}

top = 7 = the 3rd largest
```

Cost: **O(n log k)** time and **O(k)** space — better than sorting (O(n log n)) when k is small, and it works on a stream that never ends.

3.3 Heap, sort, BST or quickselect?

SITUATION	BEST TOOL	WHY
all data known, need the full order	sort, O(n log n)	simplest
all data known, need only the k-th	**quickselect**, O(n) average	no need to order anything else
need the top k, data known	heap of size k, O(n log k) — or quickselect + sort the k	
data arrives as a **stream**	heap	nothing else works incrementally
need the best, **many times, with inserts in between**	heap	its whole reason to exist
need best **and** worst, or deletions	`multiset` / two heaps	heap cannot erase
need the best, but "best" depends on a value that changes globally	heap + lazy deletion, or rethink	

Interviewers love "*can you do better than the heap?*" for LC 215. The answer is quickselect, O(n) average and O(n²) worst — and you should be able to say why the heap is still the safer choice when the input may be adversarial or streaming.

3.4 Push indices, or push enough to find your way back

As with stacks: when the heap's answer must lead you somewhere — the next element of the same list, the neighbour of a cell, the original position in an array — push the **index** (or the pointer) alongside the priority. `(dist, node)`, `(value, listIndex, posInList)`, `(freq, value)`. A bare value tells you which number is best, not where it came from.

PART 4 — The Top-K family

4.1 K-th largest element (LC 215) — three ways

```cpp
// 1. Sort: O(n log n)
sort(nums.rbegin(), nums.rend());  return nums[k - 1];

// 2. Min-heap of size k: O(n log k)
priority_queue<int, vector<int>, greater<int>> pq;
for (int x : nums) { pq.push(x); if ((int)pq.size() > k) pq.pop(); }
return pq.top();

// 3. Quickselect: O(n) average
int quickselect(vector<int>& a, int k) {          // k-th largest, 1-indexed
    int target = a.size() - k;                     // its index in ascending order
    int lo = 0, hi = a.size() - 1;
    while (true) {
        swap(a[lo + rand() % (hi - lo + 1)], a[hi]);   // random pivot
        int p = lo;
        for (int j = lo; j < hi; j++) if (a[j] < a[hi]) swap(a[p++], a[j]);
        swap(a[p], a[hi]);                         // pivot now at its sorted index p
        if (p == target) return a[p];
        if (p < target) lo = p + 1; else hi = p - 1;
    }
}
```

`std::nth_element(a.begin(), a.begin() + idx, a.end())` is the library quickselect — legitimate in an interview if you can explain what it does.

4.2 Top K frequent (LC 347, LC 692)

Count with a hash map, then run the size-k rule on `(frequency, value)` pairs.

- **LC 347** — any order is accepted. Min-heap of `(freq, value)`.
- **LC 692** — words, ordered by frequency **descending**, ties **alphabetically**. The tie-break is the whole problem: the heap must evict the *worst* candidate, which is the lower frequency, or on equal frequency the alphabetically **later** word. Write a functor and test it on a tie.

The O(n) follow-up is **bucket sort**: frequencies are at most n, so make `bucket[f]` = list of values with frequency f and walk the buckets from n down to 1.

4.3 K closest points (LC 973) and K closest elements (LC 658)

Same rule with distance as the priority. Two details:

- Compare **squared** distances — no `sqrt`, no floating point — and use `long long` if coordinates are large.
- "k closest" = "k smallest distances" → **max-heap** of size k, so the farthest kept point is on top and gets evicted.

LC 658 has a sorted input, so the optimal answer is binary search for the left edge of the window (O(log n + k)); the heap solution is the general one that works on unsorted data.

4.4 K-th largest in a stream (LC 703)

The heap is the *only* sensible structure when elements arrive one by one forever: keep the size-k min-heap as a member and answer each `add` with `top()` in O(log k).

PART 5 — K-way merge

5.1 The idea

You have k sorted sequences. The next element of the merged output must be the **smallest of the k current heads**. Keep exactly those k heads in a min-heap. Pop the smallest, output it, and push **the next element from the same sequence**.

```
merge  [1,4,5]  [1,3,4]  [2,6]

heap holds one head per list:  {1(L0), 1(L1), 2(L2)}
pop 1(L0)  push 4(L0)   heap {1(L1), 2(L2), 4(L0)}      out: 1
pop 1(L1)  push 3(L1)   heap {2(L2), 3(L1), 4(L0)}      out: 1 1
pop 2(L2)  push 6(L2)   heap {3(L1), 4(L0), 6(L2)}      out: 1 1 2
...                                                      out: 1 1 2 3 4 4 5 6
```

The heap never holds more than k entries, so each of the N total elements costs O(log k): **O(N log k)** time, **O(k)** space.

```cpp
auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
for (ListNode* h : lists) if (h) pq.push(h);           // skip empty lists!
ListNode dummy(0), *tail = &dummy;
while (!pq.empty()) {
    ListNode* n = pq.top(); pq.pop();
    tail->next = n; tail = n;
    if (n->next) pq.push(n->next);
}
```

The alternative — merge lists in pairs, divide and conquer — is also O(N log k) and is the follow-up answer when the interviewer asks for "without a heap".

5.2 Problems that are secretly k-way merges

The trick is to *see* the k sorted lists.

PROBLEM	THE K SORTED LISTS
LC 378 k-th smallest in a sorted matrix	each row
LC 373 k pairs with smallest sums	row i = `a[i]+b[0], a[i]+b[1], …`
LC 632 smallest range covering k lists	the k lists — plus track the current **max** by hand
LC 264 ugly numbers	the outputs ×2, ×3, ×5 — generate successors from each popped number
LC 355 Twitter feed	each followee's tweets, newest first
Merge k sorted arrays (GfG)	the arrays; push `(value, arrayIndex, position)`

For LC 378 and LC 373, push only the **first** element of each row (and only the first k rows), then pop k − 1 times. You never generate the n^2 candidates.

5.3 Smallest range (LC 632) — the k-way merge with a window

At any moment the heap holds one element from each list, so `[heapMin, currentMax]` is a range covering all k lists. Record it; then the only way to shrink it is to **raise the minimum**, so pop the min and push the next element of its list (updating `currentMax`). Stop when some list runs out — after that, no range covers every list.

PART 6 — Two heaps

6.1 Median of a data stream (LC 295)

Split the numbers into a **smaller half** and a **larger half**:

- `lo` — a **max-heap** holding the smaller half, so its top is the largest of the small numbers;
- `hi` — a **min-heap** holding the larger half, so its top is the smallest of the large numbers.

Keep two invariants: **every element of `lo` ≤ every element of `hi`**, and **`lo` has the same size as `hi`, or one more.** Then the median is `lo.top()` (odd count) or the average of the two tops (even count).

```
after adding 5, 15, 1, 3:

    lo (max-heap)        hi (min-heap)
    {1, 3}   top = 3     {5, 15}   top = 5         median = (3 + 5) / 2 = 4
```

The three-line `addNum` that maintains both invariants without case analysis:

```
lo.push(num);                        // 1. enter through lo
hi.push(lo.top()); lo.pop();         // 2. lo's largest moves to hi  -> order holds
if (hi.size() > lo.size()) {         // 3. rebalance                 -> sizes hold
    lo.push(hi.top()); hi.pop();
}
```

O(log n) per insert, O(1) per median.

6.2 Sliding window median (LC 480)

Now old elements must **leave** the window, and a heap cannot erase. Either:

- keep the two halves as two **multisets** (erase one copy with `erase(find(x))`), rebalancing exactly as in 6.1 — the version in the solved-questions book; or
- keep two heaps with **lazy deletion**: a hash map counts elements waiting to be deleted, and they are discarded when they reach a top. Correct but fiddly — the size bookkeeping must count only live elements.

O(n log k) either way.

6.3 Two pools: locked and available

A different two-structure pattern: candidates **unlock** over time, and among the unlocked ones you want the best.

- Sort all candidates by their **unlock condition** (capital needed, arrival time, start day).
- Walk a pointer through that sorted list, pushing everything that has unlocked into a heap ordered by **value**.
- Pop the best available.

```cpp
// IPO (LC 502): projects unlock when capital >= requirement
sort(proj.begin(), proj.end());                    // by capital needed
while (k--) {
    while (i < n && proj[i].first <= w) avail.push(proj[i++].second);
    if (avail.empty()) break;
    w += avail.top(); avail.pop();
}
```

The same skeleton is LC 1834 (Single-Threaded CPU), LC 1353 (Maximum Events), LC 871 (Refuelling Stops) and LC 1882 (Process Tasks Using Servers). Learn it once.

PART 7 — Greedy and scheduling with a heap

Many greedy algorithms say "always take the best available option." When the options change after every step, the heap is what makes that O(log n) instead of O(n).

7.1 Repeatedly combine the two best

Connect ropes (GfG), **Last Stone Weight** (LC 1046) and **Huffman coding** share one loop: pop the two best, combine them, push the result back.

```
// minimum cost to connect ropes — always join the two shortest
while (pq.size() > 1) {
    long long a = pq.top(); pq.pop();
    long long b = pq.top(); pq.pop();
    cost += a + b;
    pq.push(a + b);
}
```

Why the two shortest? Every rope's length is added to the cost once for every join it takes part in. The ropes joined first take part in the most joins, so they should be the shortest. That is exactly the Huffman argument.

7.2 Take the largest, put back what is left

LC 2558 (Take Gifts), LC 1962 (Remove Stones), LC 2208 (Halve Array Sum) and LC 1675 (Minimize Deviation) all repeatedly pop the largest, shrink it, and push it back. The heap keeps track of "the largest" as values change.

7.3 Arrange so that no two neighbours are equal

Reorganize String (LC 767): always place the most frequent remaining letter — but never the same letter twice in a row. Popping the **top two** each round and placing both guarantees that. If one letter is left with a count above 1 at the end, it is impossible (the quick check up front: impossible iff the max count exceeds ⌈n/2⌉).

Task Scheduler (LC 621) is the same idea with a cooldown of n: run the task with the most remaining copies; a task that just ran waits in a cooling **queue** until `time + n`, then returns to the heap. The closed form `max(len, (maxCount − 1)(n + 1) + numberOfTasksWithMaxCount)` is the O(n) answer; the simulation is what generalises.

7.4 The "regret" pattern — take now, undo the worst later

Some greedy problems cannot know at the moment of a decision whether it was right. The trick: **accept every option tentatively, keep the accepted ones in a heap, and when a constraint breaks, throw away the worst one.**

- **Furthest Building** (LC 1642): use a ladder on every climb; when you have used one ladder too many, pay for the **smallest** ladder-climb with bricks instead. Ladders end up on the biggest climbs.

- **Course Schedule III** (LC 630): sort by deadline, take every course; when the total time overshoots the current deadline, drop the **longest** course taken so far.
- **Refuelling Stops** (LC 871): drive past stations, remembering their fuel; when you run dry, "retroactively" stop at the **largest** station passed.
- **Maximum Performance of a Team** (LC 1383), **Minimum Cost to Hire K Workers** (LC 857): sort by one quantity, keep the best k of the other in a heap, evicting the worst.

The sentence to say: "*I commit greedily, and the heap lets me undo the single worst commitment in O(log n) when the constraint is violated.*"

7.5 Simulating machines and time

Single-Threaded CPU (LC 1834), **Process Tasks Using Servers** (LC 1882), **Meeting Rooms II** and **Maximum Events** (LC 1353) are event simulations:

- a **clock** that jumps forward when nothing can happen (never step one unit at a time through idle gaps — that is a TLE);
- a heap of **available** things ordered by the problem's rule;
- often a second heap of **busy** things ordered by the time they become free.

Use `long long` for the clock: processing times summed over 10^5 tasks overflow `int`.

PART 8 — Heaps on graphs and grids

8.1 Dijkstra's algorithm

BFS finds shortest paths when every edge costs 1, because a queue processes nodes in order of distance. With **non-negative weights**, replace the queue with a **min-heap keyed on distance** and you get Dijkstra:

```
vector<int> dist(n, INT_MAX);
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;   // (dist, node)
dist[src] = 0; pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;                 // stale entry (lazy deletion)
    for (auto [v, w] : adj[u])
        if (d + w < dist[v]) {
            dist[v] = d + w;
            pq.push({dist[v], v});             // push a NEW entry, never "update"
        }
}
```

O((V + E) log V). Three facts interviewers check:

- The **stale-entry check** replaces decrease-key. Without it the algorithm is still correct but can do far more work.
- When a node is popped with `d == dist[u]`, its distance is **final** — no later path can be shorter, because every other path leaves the heap later with a larger distance and weights are non-negative.
- With a **negative** edge that last argument fails. Use Bellman-Ford.

Topic 12 (Graphs) covers Dijkstra in depth, along with Prim's minimum spanning tree, which is the same loop keyed on edge weight instead of path length.

8.2 Minimax paths — "the cost of a path is its worst step"

Swim in Rising Water (LC 778), **Path With Minimum Effort** (LC 1631): the cost of a path is its **maximum** cell or step, not its sum. Dijkstra still works — push `max(currentCost, cellCost)` instead of `currentCost + edgeWeight`. The first time the target is popped, its cost is optimal.

8.3 Trapping Rain Water II (LC 407)

In 2-D, the water over a cell is limited by the **lowest** point of the wall around it. Push every border cell into a min-heap. Pop the lowest wall cell; for each unvisited neighbour, it can hold `max(0, wallHeight − neighbourHeight)` water, and it joins the wall at height `max(wallHeight, neighbourHeight)`. The wall shrinks inward from its lowest point, which is exactly where water would spill.

This is the 2-D generalisation of LC 42 and a favourite "hard" in on-campus drives that want to see whether you can generalise a two-pointer idea.

PART 9 — The complete pattern taxonomy

Every problem in Part 13 is labelled with one of these twelve. When you can name the pattern, you already have most of the code.

#	PATTERN	RECOGNISE IT BY	CORE TOOL
1	Repeatedly take the best	"each turn, take the largest/smallest and …"	heap, pop / modify / push
2	K-th / top-K with a bounded heap	"k-th largest", "top k", "k closest"	min-heap of size k for largest (and vice versa)
3	Top-K by frequency	"k most frequent"	hash map counts → heap of `(freq, value)`
4	K-way merge	"merge k sorted", sorted matrix, smallest pairs	one head per list, push the successor
5	Two heaps	median, "balance two halves"	max-heap low half + min-heap high half
6	Sort + heap of available options	options unlock over time (capital, arrival, start day)	pointer over sorted input + value heap
7	Greedy combine the two best	connect ropes, stones, Huffman	pop two, push the combination
8	Regret / exchange greedy	"take as many as possible under a limit"	accept all, evict the worst when the limit breaks
9	Scheduling with cooldown or no-adjacent rule	task scheduler, reorganize string	heap of counts + a waiting queue / hold one back
10	Event simulation	CPU, servers, meeting rooms	a clock that jumps + heaps of free and busy
11	Dijkstra-style search	weighted shortest path, minimax path, 2-D water	min-heap of `(cost, state)` + stale-entry skip
12	Heap with deletions	sliding window median, skyline	`multiset`, or lazy deletion

How to tell 2 and 4 apart

Both find "the k-th something".

- **Pattern 2** — the candidates are **given all at once**, unordered. Stream them through a heap of size k.
- **Pattern 4** — the candidates are **implied by sorted structure** (rows of a matrix, pairs of two sorted arrays) and there are too many to list. Push only the head of each sorted run and generate successors lazily.

If listing every candidate would be $O(n^2)$ or worse, you are in 4.

How to tell 6 and 8 apart

- **Pattern 6** asks "what is the best option **available now**?" — nothing you took is ever given back.
- **Pattern 8** asks "which of the options I **already took** should I give back?" — the heap holds your past choices, not your future ones.

PART 10 — How to approach ANY heap problem

Six steps. They prevent the three expensive mistakes: the wrong heap direction, pushing too much into the heap, and a simulation that steps through idle time.

Step 1 — Ask whether you need "the best", repeatedly, while the set changes

If you need it once → scan. If the set never changes → sort. If you need the k-th of a fixed array → consider quickselect. Only if the answer is "repeatedly, with inserts in between" (or a stream) is a heap clearly right.

Step 2 — Decide what one heap entry is

Write the tuple down: `(priority, payload)`. The payload is whatever lets you **act** on the popped element — an index, a list pointer, `(row, col)`, a node id. Priority goes first so `pair` / `tuple` ordering does the comparing for you.

Step 3 — Decide the direction, out loud

Say: "*I pop the smallest distance, so it is a min-heap.*" or "*I keep the k largest, so the top must be the weakest of them — a min-heap.*" Then write `greater<>` or not. This is the step people skip and the bug people make most.

Step 4 — Bound the heap

Can the heap hold only k entries (top-K) or only one per list (k-way merge)? Bounding it changes O(n log n) into O(n log k). If entries go stale (Dijkstra, skyline), decide where you skip them.

Step 5 — Handle the clock and the empty heap

For simulations: when the heap of available items is empty, **jump** the clock to the next event. Before every `top()`: is the heap guaranteed non-empty here? If not, guard it.

Step 6 — Dry-run the small cases

1. **Empty input**, or k = 0 if the problem allows it.
2. **k = 1** and **k = n**.
3. **All equal priorities** — this is where tie-break rules (LC 692, LC 1337, LC 658) are tested.
4. **Already sorted** and **reverse sorted** input.
5. For simulations: a **gap** in time where nothing is available.

PART 11 — The bug catalogue

1. Wrong heap direction. `priority_queue<int>` is a **max**-heap. Writing it when you meant a min-heap (Dijkstra, k-way merge, k largest) gives wrong answers that often *look* plausible. Say the direction out loud before typing.

2. Sifting down with the smaller child. In a max-heap, swap with the **larger** child; in a min-heap, with the smaller. Symptom: the hand-built heap passes small tests and pops out of order on larger ones.

3. Not checking that the right child exists. `2*i + 2 < n` must be checked separately from `2*i + 1 < n`. Symptom: reading past the heap's end — garbage values in heap sort, where the array beyond `n` holds already-sorted elements.

4. The comparator written the `std::sort` way. `cmp(a, b)` returns true when `a` goes **below** `b`. A comparator that returns `a.dist < b.dist` builds a **max**-heap on distance. Symptom: you get the farthest points instead of the closest.

5. Calling `top()` or `pop()` on an empty heap. Undefined behaviour. Symptom: crash or garbage on the judge only. Common when popping "two at a time" (LC 1046, LC 767) — check `size() >= 2`.

6. Expecting `pop()` to return the value. It is `void`. Read `top()` first.

7. Pushing everything when only k are needed. Building a heap of n^2 pairs for LC 373 / LC 378. Symptom: MLE or TLE. Push one head per sorted run and generate successors.

8. Forgetting to skip stale entries. In Dijkstra without `if (d > dist[u]) continue;`, each stale pop re-relaxes all edges. Symptom: correct answers, TLE on dense graphs.

9. `multiset::erase(value)` instead of `erase(find(value))`. The first erases **every** copy. Symptom: the window shrinks too fast when it contains duplicates.

10. Integer overflow in the priority. Squared distances (`x*x + y*y`), sums of costs (connect ropes), `speedSum * efficiency`, a simulation clock. Symptom: wrong answers only on large inputs. Use `long long` in the heap's type, not just in the final variable.

11. Stepping the clock one unit at a time through idle gaps. Symptom: TLE when times are up to 10^9. Jump the clock to the next arrival when nothing is available.

12. Pushing empty lists in a k-way merge. `pq.push(lists[i])` where `lists[i]` is `nullptr`, then dereferencing it in the comparator. Symptom: crash on `[[]]`.

13. Ignoring the tie-break. LC 692 wants alphabetical order on equal frequency; LC 1337 wants the smaller index; LC 658 wants the smaller value. The free tie-break of `pair` may be the opposite one. Symptom: fails only on test cases with ties.

14. Taking the modulo before comparing. LC 1383 asks for the maximum performance, *returned* modulo $10^9 + 7$. Taking `% MOD` inside the loop compares reduced numbers. Symptom: wrong answer on large inputs. Maximise in `long long`, reduce once at the end.

PART 12 — Complexity cheat sheet

OPERATION / ALGORITHM	TIME	SPACE	NOTE
push, pop	O(log n)	–	one root-to-leaf path
top, size, empty	O(1)	–	
build-heap (heapify)	**O(n)**	O(1)	not O(n log n) — Part 1.3
n pushes one by one	O(n log n)	O(n)	
heap sort	O(n log n)	**O(1)**	not stable
k-th largest, heap of size k	O(n log k)	O(k)	
k-th largest, quickselect	O(n) average, $O(n^2)$ worst	O(1)	`nth_element`
top-k frequent	O(n log k)	O(n)	O(n) with bucket sort
merge k sorted lists (N total)	O(N log k)	O(k)	
k-th smallest in sorted n×n matrix	O(k log n)	O(n)	binary search on value is O(n log(range))
median of a stream	O(log n) add, O(1) median	O(n)	two heaps
sliding window median	O(n log k)	O(k)	multisets or lazy deletion
Dijkstra	O((V + E) log V)	O(V + E)	non-negative weights
`multiset` insert / erase / min / max	O(log n)	O(n)	the heap you can erase from

The sentences to say in interviews:

> *"A min-heap of size k holds the k largest elements seen so far; its top is the k-th largest. Each element costs O(log k), so it is O(n log k) overall and works on a stream."*
>
> *"Building a heap bottom-up is O(n), not O(n log n), because most nodes are near the leaves and sift down only a level or two."*
>
> *"Instead of decrease-key I push a new entry and skip stale ones when they are popped — lazy deletion."*

PART 13 — The problem list

Work these in order. Every one is solved in the companion **Heaps & Priority Queues — Solved Questions** PDF, in exactly this order — but attempt each problem yourself first and only read the solution after you have genuinely struggled with it.

Tier 1 — Learn the mechanics (do all thirteen first)

#	PROBLEM	PATTERN
1046	Last Stone Weight	7 — combine the two best
703	Kth Largest Element in a Stream	2 — bounded heap, streaming
215	Kth Largest Element in an Array	2 — bounded heap ⭐
347	Top K Frequent Elements	3 — top-K by frequency ⭐
973	K Closest Points to Origin	2 — max-heap of the k closest ⭐
1337	The K Weakest Rows in a Matrix	2 — tie-break by index
2558	Take Gifts From the Richest Pile	1 — take the best, put back
506	Relative Ranks	1 — pop in order
451	Sort Characters By Frequency	3 — frequency heap
692	Top K Frequent Words	3 — custom comparator, ties
23	Merge k Sorted Lists	4 — k-way merge ⭐
378	Kth Smallest Element in a Sorted Matrix	4 — rows as sorted lists
1962	Remove Stones to Minimize the Total	1 — take the best, put back

Thirteen problems. When you can write 215, 347, 973 and 23 from a blank page — and say the heap direction for each without hesitating — you have the mechanics.

Tier 2 — The placement core (these are what actually get asked)

#	PROBLEM	PATTERN
295	Find Median from Data Stream	5 — two heaps ⭐
621	Task Scheduler	9 — cooldown queue ⭐
767	Reorganize String	9 — take the top two ⭐
1642	Furthest Building You Can Reach	8 — regret greedy
1834	Single-Threaded CPU	10 — event simulation
373	Find K Pairs with Smallest Sums	4 — implied sorted rows
355	Design Twitter	4 — merge followees' feeds
264	Ugly Number II	4 — generate successors
658	Find K Closest Elements	2 — distance with a tie-break
846	Hand of Straights	1 — smallest must start a run
743	Network Delay Time	11 — Dijkstra ⭐
1353	Maximum Number of Events That Can Be Attended	6 — earliest deadline among available
1405	Longest Happy String	9 — hold one back
871	Minimum Number of Refueling Stops	8 — retroactive greedy
1882	Process Tasks Using Servers	10 — free and busy heaps

⭐ **295 Median from Data Stream, 23 Merge k Sorted Lists, 215 Kth Largest, 347 Top K Frequent, 973 K Closest Points, 621 Task Scheduler and 743 Network Delay Time are the most-asked heap questions in on-campus drives** — Amazon, Microsoft, Adobe, Flipkart, Walmart, Goldman Sachs. Be able to write all of them cold.

Tier 3 — Harder, but they do appear

#	PROBLEM	PATTERN
480	Sliding Window Median	12 — two halves with deletion
502	IPO	6 — locked vs available
630	Course Schedule III	8 — drop the longest
857	Minimum Cost to Hire K Workers	8 — sort by ratio, evict the heaviest
1383	Maximum Performance of a Team	8 — sort by efficiency, evict the slowest
632	Smallest Range Covering Elements from K Lists	4 — k-way merge + running max
407	Trapping Rain Water II	11 — the wall shrinks from its lowest point
778	Swim in Rising Water	11 — minimax Dijkstra
1675	Minimize Deviation in Array	1 — normalise, then shrink the max
218	The Skyline Problem	12 — lazy deletion

The classic Indian-placement questions (GfG / written-round favourites)

These appear in written rounds and technical interviews, usually in GfG's wording:

- **Implement a max-heap / min-heap** with insert, extract-max and heapify — on an array
- **Build a heap from an array** — and state why it is O(n)
- **Heap sort** — write it; is it stable? what is its space?
- **Check whether an array represents a binary max-heap**
- Convert a min-heap into a max-heap
- K largest / k smallest elements in an array
- **K-th smallest element** — heap and quickselect
- **Sort a nearly sorted (k-sorted) array** in O(n log k)
- **Merge k sorted arrays**
- **Minimum cost of ropes** (connect n ropes)
- **Median in a stream** (running median)
- Top k frequent elements; k most frequent words
- Rearrange characters so no two adjacent are the same
- Kth largest element in a stream
- Smallest range in k lists
- Huffman encoding — build the tree with a min-heap
- Dijkstra's algorithm with a priority queue
- **Is a binary tree a heap?** — completeness plus the heap property
- Maximum sum combinations from two arrays (the LC 373 idea, largest instead of smallest)

"Build a heap from an array — what is the complexity?" is close to a guaranteed written-round question. The expected answer is **O(n)**, with the argument from Part 1.3: half the nodes are leaves and do no work, a quarter sift down one level, and the sum n · Σ h / 2^(h+1) is bounded by n. Saying "O(n log n)" is the trap.

PART 14 — A three-week plan

Assumes about 1.5 hours a day. The heap is a small structure with a large number of uses — week 1 is the structure, weeks 2 and 3 are the uses.

Week 1 — The structure. Days 1–2: Parts 0–1, then practice TODOs 1–6 **without** `std::priority_queue`: `isMaxHeap`, sift-up, sift-down, the `MaxHeap` class, build-heap, heap sort. Write build-heap's O(n) argument out on paper. Day 3: Part 2 — write a min-heap three ways (`greater<>`, a functor, a lambda) and convince yourself of the comparator rule. Days 4–7: Tier 1 — 1046, 703, 215, 347, 973, 1337, 2558, 506, 451, 692; practice TODOs 7–9.

Week 2 — The four big patterns. K-way merge (23, 378, 373, 264, 355). Two heaps (295 and TODO 10, then 480). Sort + available heap (1353, 1834, 502). Dijkstra (743). Before each problem, say out loud what one heap entry is and which direction the heap points.

Week 3 — Greedy and hard. 621, 767, 1405 in one sitting — they are one idea. Then the regret pattern: 1642, 871, 630, 1383, 857. Then 632, 407, 778, 218 as far as you get. Finish by re-solving **295, 23, 215, 347, 973 and 621** on a timer — 15 minutes each, on paper, no compiler.

Ongoing: every time a problem defeats you, log which of the 12 patterns it was and which of the bugs you hit. After thirty problems that log tells you exactly what to drill.

Where this topic leads: Dijkstra and Prim are the centre of Topic 12 (Graphs). The regret pattern returns in Topic 10 (Greedy), where exchange arguments prove it correct. Heap sort and quickselect belong to Topic 09 (Searching & Sorting). The LRU and LFU caches in Topic 15 are what you build when a heap is *not* enough. Nothing here is wasted.

Quick reference card

```
// ===== index formulas (0-indexed) =====
parent(i) = (i - 1) / 2;   left(i) = 2 * i + 1;   right(i) = 2 * i + 2;
// leaves: i >= n/2      last internal node: n/2 - 1      height: floor(log2 n)

// ===== sift down (max-heap), used by pop / heapify / heap sort =====
while (true) {
    int big = i, l = 2*i + 1, r = 2*i + 2;
    if (l < n && a[l] > a[big]) big = l;
    if (r < n && a[r] > a[big]) big = r;
    if (big == i) break;
    swap(a[i], a[big]); i = big;
}

// ===== build-heap O(n), heap sort =====
for (int i = n/2 - 1; i >= 0; i--) siftDown(a, n, i);
for (int end = n - 1; end > 0; end--) { swap(a[0], a[end]); siftDown(a, end, 0); }

// ===== STL =====
priority_queue<int> maxH;                                    // largest on top
priority_queue<int, vector<int>, greater<int>> minH;         // smallest on top
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;   // min by .first
auto cmp = [](const T& a, const T& b) { return a.key > b.key; };     // '>' = min-heap
priority_queue<T, vector<T>, decltype(cmp)> pq2(cmp);
// cmp(a, b) == true  means  a sits BELOW b.   pop() returns void.

// ===== k largest: MIN-heap of size k =====
for (int x : a) { minH.push(x); if ((int)minH.size() > k) minH.pop(); }
// minH.top() is the k-th largest

// ===== k-way merge =====
for (each list i) if (nonEmpty) pq.push({head value, i, 0});
while (!pq.empty()) { auto [v, i, j] = pq.top(); pq.pop(); out(v);
                      if (j + 1 < len(i)) pq.push({list[i][j+1], i, j+1}); }

// ===== two heaps: running median =====
lo.push(x); hi.push(lo.top()); lo.pop();
if (hi.size() > lo.size()) { lo.push(hi.top()); hi.pop(); }
median = lo.size() > hi.size() ? lo.top() : (lo.top() + (double)hi.top()) / 2;

// ===== sort + available heap =====
sort(items by unlock time);
while (...) { while (i < n && unlocked(items[i])) avail.push(items[i++]);
              if (avail.empty()) { jump clock; continue; }  take(avail.top()); avail.pop(); }

// ===== Dijkstra =====
auto [d, u] = pq.top(); pq.pop();
if (d > dist[u]) continue;                                   // stale
for (auto [v, w] : adj[u]) if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }

// ===== deletable heap =====
multiset<int> s;  *s.begin();  *s.rbegin();  s.erase(s.find(x));   // ONE copy
```

The five rules, one last time:

1. **A heap answers one question cheaply: "what is the best right now?"** Use it when you ask that repeatedly while the set changes.
2. Decide **what one entry is** and **which way the heap points** before writing code — and say it out loud.
3. **k largest → min-heap of size k. k smallest → max-heap of size k.** The top is the one to evict.

4. `cmp(a, b)` true means **a goes below b**. `priority_queue<int>` is a **max**-heap.
5. A heap cannot erase or update: **push a new entry and skip stale ones**, or use a `multiset`.